# MINI PROJECT REPORT

On

## "EduVault: Digital Locker and Academic Credential Verification Platform"

**By**

**Akshay Darji** — PRN: **1272250311**

**Priyesh Nandanwar** — PRN: **1272250279**

For

**MCA40130 Mini Project**

**SY MCA (Semester III)**

**AY 2026-2027**

Under The Guidance Of

**Mrs. Sukanya Akolkar**

Submitted To

**Dr. Vishwanath Karad**

**MIT WORLD PEACE UNIVERSITY, PUNE**

In partial fulfillment of the requirement for the award of Degree of

**Master of Computer Applications (MCA)**

Submitted Through

**School of Computer Science and Engineering**

**Department of Computer Science and Applications**

MIT-WPU, Pune - 411038

**Dr. Vishwanath Karad MIT WORLD PEACE UNIVERSITY, PUNE**

School of Computer Science and Engineering

**Department of Computer Science and Applications**

# CERTIFICATE

This is to certify that, **Akshay Darji** (PRN: **1272250311**) and **Priyesh Nandanwar** (PRN: **1272250279**), students of **SY MCA, Semester III** have completed Mini Project entitled **"EduVault: Digital Locker and Academic Credential Verification Platform"** under guidance of **Mrs. Sukanya Akolkar** in partial fulfilment of requirement for the award of **Master of Computer Applications** prescribed under **Dr. Vishwanath Karad MIT World Peace University, Pune**.

This project is the record of authentic work carried out by them during the academic year **2026-2027**.

**Mrs. Sukanya Akolkar**
Internal Guide

**Dr. P. S. Metkewar**
Program Head - MCA

**Dr. Anuradha S. Kanade**
Program Director

**Date:** 27 / 10 / 2026 **Place:** Pune

# ACKNOWLEDGEMENT

I wish to express our profound gratitude to all the individuals who supported us throughout the completion of our Mini Project III. This report is a testament to their guidance, patience, and resources.

We are deeply indebted to our esteemed project guide, **Mrs. Sukanya Akolkar**, for their invaluable mentorship, direction, and encouragement. Their advice was crucial in defining the scope and methodology of this project.

We extend our sincere thanks to the Head of the Program, **Dr. P. S. Metkewar**, and the Program Director, **Dr. Anuradha S. Kanade**, for providing the necessary facilities, academic framework, and environment conducive to advanced project work and research.

Our appreciation also goes to all the **Faculty Members of the Department of Computer Science and Applications** for their comprehensive instruction and expertise, which equipped us with the knowledge foundational to this project.

We thank our fellow Project Group Members, for their unwavering dedication, collaborative spirit, and commitment to shared goals. This project is a product of our collective effort.

Finally, we are grateful to our **Classmates** for their constructive feedback and discussions, which enriched our understanding and helped us refine our approach.

| Field | Member 1 | Member 2 |
|---|---|---|
| **Name:** | Akshay Darji | Priyesh Nandanwar |
| **PRN:** | 1272250311 | 1272250279 |
| **Signature:** | | |



**Date:** 27 / 10 / 2026 **Place:** Pune

# DECLARATION

We, **Akshay Darji** and **Priyesh Nandanwar**, on behalf of our team hereby declare that this Mini Project report entitled **"EduVault: Digital Locker and Academic Credential Verification Platform"** is an authentic record of our own work carried out under the supervision of **Mrs. Sukanya Akolkar**.

We further declare that the content of this report, including all design, coding, testing, and analysis, is original and has not been submitted previously for any other degree or diploma at this or any other institution.

We take sole and full responsibility for any errors, omissions, or misrepresentations found within this report.

| Field | Member 1 | Member 2 |
|---|---|---|
| **Name:** | Akshay Darji | Priyesh Nandanwar |
| **PRN:** | 1272250311 | 1272250279 |
| **Signature:** | | |

# PROJECT REPORT INDEX


| Index | Section Title | Page |
|---|---|---|
| 1 | **INTRODUCTION** | 1 |
| 1.1 | EXISTING SYSTEM, PROPOSED SYSTEM AND NEED AND OBJECTIVE OF SYSTEM | 1 |
| 1.2 | SCOPE OF WORK | 3 |
| 1.3 | OPERATING ENVIRONMENT – HARDWARE AND SOFTWARE | 4 |
| 1.4 | DESCRIPTION OF TECHNOLOGY USED | 5 |
| 2 | **PROPOSED SYSTEM** | 7 |
| 2.1 | FEASIBILITY STUDY | 7 |
| 2.2 | USER REQUIREMENTS | 9 |
| 3 | **ANALYSIS AND DESIGN (If Applicable)** | 11 |
| 3.1 | OBJECT DIAGRAM | 11 |
| 3.2 | CLASS DIAGRAM | 12 |
| 3.3 | USE CASE DIAGRAM | 14 |
| 3.4 | SEQUENCE DIAGRAM | 15 |
| 3.5 | ACTIVITY DIAGRAM | 17 |
| 3.6 | MODULE HIERARCHY DIAGRAM | 18 |
| 3.7 | USER INTERFACE DESIGN (SCREENS ETC.) | 19 |
| 3.8 | TABLE SPECIFICATIONS | 21 |
| 4 | **INPUT AND OUTPUT SCREENS** | 24 |
| 5 | **TEST PROCEDURES AND IMPLEMENTATION** | 28 |
| 6 | **SAMPLE CODE** | 32 |

| | | |
|---|---|---|
| 7 | LIMITATIONS | 36 |
| 8 | FUTURE SCOPE | 37 |
| 9 | CONCLUSION | 38 |
| 10 | REFERENCES | 39 |


# 1. INTRODUCTION

## 1.1 Existing System, Proposed System and Need and Objective of System

**Existing System:** Traditional academic credential management in higher education institutes heavily depends on physical paper documents or unauthenticated PDF files. In this manual model, students submit physical photocopies for placement eligibility and administrative checks. Academic verification committees manually inspect and stamp records, leading to severe administrative overhead, multi-week delays for corporate background checks, and an inherent vulnerability to resume fraud and document alteration.

**Proposed System (EduVault):** EduVault is an enterprise-grade Digital Locker and Academic Credential Verification platform. It provides a secure, role-governed ecosystem supporting four primary user roles: *Student*, *Alumni*, *Academic Committee*, and *Placement Team*. Students securely upload marksheets, certificates, internships, and technical projects. The Academic Committee reviews and verifies credentials in real-time. Placement officers leverage verified student dossiers with dynamic CGPA and departmental filtering. Every candidate receives a cryptographic **Interview Key** that compiles into an instant QR Code for on-the-spot mobile verification by recruiters.

**Need of the System:** There is a critical institutional need to eradicate academic fraud, streamline campus placement drives, and offer students lifelong cloud-native access to authenticated academic credentials. EduVault provides an immutable, transparent single source of truth for students, recruiters, and administrators.

**Objectives of the System:**

- Implement a high-speed, secure RESTful API using Django REST Framework and PostgreSQL.
- Deliver an intuitive, responsive frontend using React 18, Vite, and Tailwind CSS.
- Establish strict Role-Based Access Control (RBAC) across 4 roles.
- Integrate dynamic QR Code generation for instant physical and digital interview verification.

- Deploy the entire full-stack platform to 100% free production cloud tiers (Vercel, Render, and Neon).

### 1.2 Scope of Work

The scope of EduVault covers user authentication with JWT, digital document storage with MIME-type and size validators, an academic committee approval/rejection pipeline with candidate feedback, a placement candidate discovery portal, a dynamic alumni directory with mentorship status, and public endpoints for QR-based dossier verification.

### 1.3 Operating Environment – Hardware and Software

**Hardware Requirements:** Dual-core 2.0 GHz processor or higher, 8 GB RAM (16 GB recommended), 500 MB storage. Compatible with standard desktop browsers and modern Android/iOS mobile cameras.

**Software Requirements:** Windows 11 / Ubuntu 22.04 LTS, Python 3.11, Node.js v20, Django 6.0, Django REST Framework, React 18, Vite, PostgreSQL 16 (Neon Serverless), WhiteNoise, and Gunicorn.

### 1.4 Description of Technology Used

**React 18 & Vite:** High-performance reactive UI rendering with instantaneous hot module reloading and fast client bundle compilation.

**Tailwind CSS:** Modern utility-first CSS styling enabling responsive layouts, micro-animations, and clean typography without heavy style dependencies.

**Django & Django REST Framework:** Robust Python backend framework providing built-in ORM security against SQL injections, CSRF protection, and high-throughput serialization.

**Neon PostgreSQL 16:** Cloud-native serverless PostgreSQL providing full ACID compliance, foreign key integrity, and zero-maintenance storage.

---

## 2. PROPOSED SYSTEM

### 2.1 Feasibility Study

**Operational Feasibility:** The interface eliminates manual paperwork. Students can upload records in seconds; committee members review them with single-click approvals; placement officers filter candidates instantly. The system is operationally feasible and requires zero special training.

**Technical Feasibility:** Built using proven, robust technologies (Python, Django, React, PostgreSQL) backed by extensive documentation and active developer communities. The system is technically sound.

**Economic Feasibility:** EduVault utilizes 100% open-source software and is hosted on zero-cost production cloud infrastructure (Vercel, Render, and Neon), resulting in zero software licensing or operational hosting costs.

### 2.2 User Requirements

**Functional Requirements:** Secure registration & JWT login; role-based dashboard redirection; document uploading (PDF/JPG/PNG <= 5MB); committee verification & rejection with feedback; placement search with CGPA filtering; dynamic alumni registry; QR code public dossier generation.

**Non-Functional Requirements:** Sub-300ms API response time; HTTPS encrypted communication; PBKDF2 password hashing with SHA-256; WCAG-compliant responsive design; persistent cloud database integrity.

---

## 3. ANALYSIS AND DESIGN

### 3.1 Object Diagram



```
  - id: 42                                              - id: 3
  - title: "Degree Certificate"                         - role: "COMMITTEE"
  - status: "VERIFIED"
  - verification_id: "f47ac10b..."
```

## 3.2 Class Diagram

```
+------------------------+          1:1          +--------------------------+
|         User           |----------------------->|      StudentProfile      |
+------------------------+                        +--------------------------+
| - id: int              |                        | - id: int                |
| - email: string        |                        | - prn: string            |
| - role: string         |                        | - cgpa: float            |
| - college: string      |                        | - batch: string          |
| - department: string   |                        | - current_company: string|
+------------------------+                        +--------------------------+
            | 1:*
            |
            v
+------------------------+
|       Document         |
+------------------------+
| - id: int              |
| - verification_id: uuid|
| - title: string        |
| - category: string     |
| - file: FileField      |
| - status: string       |
| - rejection_reason: str|
+------------------------+
```

## 3.3 Use Case Diagram

**Primary Actors:** Student, Alumni, Committee Member, Placement Officer, Public Recruiter.
**Core Use Cases:** Register & Login, Manage Profile, Upload/Delete Documents, Generate Interview Key QR, Review Pending Queue, Verify/Reject Document, Filter Candidates by CGPA, Browse Alumni Network, Scan QR & Verify Dossier.

## 3.4 Sequence Diagram (Document Verification & Recruitment Flow)

```
Student -> ReactUI: Uploads Degree Certificate (PDF)
ReactUI -> DjangoAPI: POST /api/documents/ (multipart + JWT)
```

```
DjangoAPI -> NeonDB: INSERT INTO documents (status="PENDING")
DjangoAPI -> ReactUI: 201 Created (Badge: PENDING)

Committee -> ReactUI: Opens Verification Queue
ReactUI -> DjangoAPI: GET /api/committee/documents/?status=PENDING
Committee -> ReactUI: Clicks "Verify Document"
ReactUI -> DjangoAPI: POST /api/committee/documents/{id}/verify/
DjangoAPI -> NeonDB: UPDATE documents SET status="VERIFIED"
DjangoAPI -> ReactUI: 200 OK (Badge: VERIFIED)

Recruiter -> PhoneCamera: Scans Student's Interview Key QR Code
PhoneCamera -> ReactUI: Opens /student/14
ReactUI -> DjangoAPI: GET /api/profile/public/student/14/
DjangoAPI -> ReactUI: Returns Verified Dossier with Approved Documents
ReactUI -> Recruiter: Displays Authenticated Candidate Credentials
```

### 3.5 Activity Diagram

User logs in → Selects document & category → Validates size & extension → Submits to Django REST API → Backend saves record with status *PENDING* → Document enters Committee Queue → Committee inspects file → If approved: status updated to *VERIFIED* with timestamp; If rejected: status updated to *REJECTED* with feedback dispatched to student → Verified documents compile into student's public Interview Key.

### 3.6 Module Hierarchy Diagram

**EduVault Platform** decomposed into: (1) Authentication & RBAC Module, (2) Student Digital Vault Module, (3) Academic Committee Verification Module, (4) Placement Talent Pool Module, (5) Dynamic Alumni Directory Module, and (6) Public Verification Gateway.

### 3.7 User Interface Design (Screens etc.)

Consists of 8 responsive screens: User Login, User Registration, Student Digital Locker, Interview Key QR Modal, Academic Committee Review Queue, Campus Placement Talent Pool, Dynamic Alumni Directory, and Public Verified Candidate Dossier.

### 3.8 Table Specifications

**Table 1: accounts_user** — Columns: `id` (PK), `email` (Unique), `password` (Hash), `role` (STUDENT, ALUMNI, COMMITTEE, PLACEMENT), `college`, `department`, `is_active`, `date_joined`.

**Table 2: students_studentprofile** — Columns: `id` (PK), `user_id` (FK 1:1), `prn`, `cgpa`, `batch`, `skills`, `current_company`, `current_role`, `is_mentor_available`.

**Table 3: documents_document** — Columns: `id` (PK), `verification_id` (UUID), `user_id` (FK), `title`, `category`, `file`, `status` (PENDING, VERIFIED, REJECTED), `rejection_reason`, `verified_at`.

---

## 4. INPUT AND OUTPUT SCREENS

Detailed documentation of all system screens, input fields, button interactions, visual indicators, and feedback messages across the 4 user portals.

---

## 5. TEST PROCEDURES AND IMPLEMENTATION

| Test ID | Scenario | Input | Expected Result | Status |
|---|---|---|---|---|
| TC-01 | User Registration | Akshay Darji, ak12@gmail.com, STUDENT | User created, 201 Created | **PASS** |
| TC-02 | Duplicate Email Reg | ak12@gmail.com (duplicate) | HTTP 400 Bad Request | **PASS** |
| TC-03 | User Login | Valid Email & Password | Return JWT Access & Refresh | **PASS** |
| TC-04 | Document Upload Valid | marksheet.pdf (1.8 MB), MARKSHEET | Saved with status PENDING | **PASS** |
| TC-05 | Oversized Upload | file > 5 MB | HTTP 400 File size error | **PASS** |
| TC-06 | Role Access Guard | Student accessing Committee API | HTTP 403 Forbidden | **PASS** |
| TC-07 | Committee Approval | Document ID: 42 clicked Verify | Status updated to VERIFIED | **PASS** |
| TC-08 | Placement CGPA Filter | CGPA >= 8.0 | Filtered candidate pool | **PASS** |
| TC-09 | Mobile QR Scan | Phone camera scans /student/14 | Opens public dossier on phone | **PASS** |

| | | | | |
|---|---|---|---|---|
| TC-10 | Production Media View | Click "View File" | Opens file from Render backend | **PASS** |

---

## 6. SAMPLE CODE

### Custom User Model (`accounts/models.py`):

```
class User(AbstractUser):
    class Role(models.TextChoices):
        STUDENT = "STUDENT", "Student"
        ALUMNI = "ALUMNI", "Alumni"
        COMMITTEE = "COMMITTEE", "Committee Member"
        PLACEMENT = "PLACEMENT", "Placement Team"

    username = None
    email = models.EmailField(unique=True)
    role = models.CharField(max_length=20, choices=Role.choices,
default=Role.STUDENT)
    college = models.CharField(max_length=255, default="MIT-WPU")
    department = models.CharField(max_length=100, default="Computer Science")
    USERNAME_FIELD = "email"
    REQUIRED_FIELDS = ["first_name", "last_name"]
```

### Committee Verification API View (`documents/views.py`):

```
class VerifyDocumentView(APIView):
    permission_classes = [IsCommittee]

    def post(self, request, pk):
        document = get_object_or_404(Document, pk=pk)
        document.status = Document.Status.VERIFIED
        document.verified_at = timezone.now()
        document.rejection_reason = None
        document.save()
        return Response({"message": "Document verified successfully.", "status":
"VERIFIED"})
```

### Frontend Dynamic Media Resolver (`src/api/apiClient.js`):

```
export const API_BASE_URL = typeof import.meta !== "undefined" &&
import.meta.env?.VITE_API_URL
```

```
  ? import.meta.env.VITE_API_URL.replace(/\/$/, "") : "";

export const getMediaUrl = (url) => {
  if (!url) return "";
  if (url.startsWith("http://") || url.startsWith("https://")) return url;
  const cleanPath = url.startsWith("/") ? url : `/${url}`;
  return `${API_BASE_URL}${cleanPath}`;
};
```

---

## 7. LIMITATIONS

1. Render free-tier cold starts (30-50s spin up after 15 min idle).
2. Ephemeral disk on free container restarts (persistent PostgreSQL, temporary media).
3. Manual committee verification without automated OCR verification.
4. Offline mode not currently available (requires active internet).

## 8. FUTURE SCOPE

1. Blockchain-anchored cryptographic certificate hashes on Ethereum / Polygon.
2. Integration with persistent AWS S3 or Cloudinary storage.
3. AI-powered OCR for automatic verification against university ERP databases.
4. Government DigiLocker and National Academic Depository (NAD) API integrations.
5. Native mobile applications built with React Native.

---

## 9. CONCLUSION

EduVault successfully modernizes academic credential handling, eliminates document fraud, simplifies administrative review, and bridges the gap between students, recruiters, and alumni. Built on Django REST Framework, React 18, Vite, and PostgreSQL, the platform is fully responsive, production-tested, and deployed on cloud infrastructure.

## 10. REFERENCES

1. Daniel Roy Greenfeld & Audrey Roy Greenfeld, *Two Scoops of Django 3.x*.
2. Alex Banks & Eve Porcello, *Learning React (2nd Edition)*, O'Reilly Media.
3. Django Software Foundation, *Django 6.0 Documentation*, 2026.

4. Django REST Framework Official Guide, *https://www.django-rest-framework.org/*.
5. React.js Documentation, *https://react.dev/*.
6. Vite Documentation, *https://vite.dev/*.
7. Tailwind CSS Guide, *https://tailwindcss.com/docs*.
8. PostgreSQL Global Development Group, *PostgreSQL 16 Manual*, 2026.